

## 10.4 The singular value decomposition

QR is the workhorse when  $\mathbf{X}$  has full column rank. When it does not, or nearly does not, the singular value decomposition tells the whole story: every least squares solution, the one of minimum norm, and, in the smallest singular values, the combinations of the columns that the data cannot resolve. Theorem 1.8.1 established the decomposition, and Proposition 6.10.1 previewed its use for least squares.

### 10.4.1 Least squares through the SVD

Throughout,  $\mathbf{X}$  is  $n \times p$  of rank  $r \geq 1$ , with singular value decomposition  $\mathbf{X} = \sum_{i \leq r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$  (Theorem 1.8.1). We complete  $\mathbf{u}_1, \dots, \mathbf{u}_r$  and  $\mathbf{v}_1, \dots, \mathbf{v}_r$  to orthonormal bases  $\mathbf{u}_1, \dots, \mathbf{u}_n$  of  $\mathbb{R}^n$  and  $\mathbf{v}_1, \dots, \mathbf{v}_p$  of  $\mathbb{R}^p$ , and write  $c_i = \mathbf{u}_i^\top \mathbf{y}$  for the coordinates of the data in the left singular basis.

#### Theorem 10.4.1 — Least squares via the SVD

- (a)  $\mathbf{b}$  is a least squares estimate iff  $\mathbf{v}_i^\top \mathbf{b} = c_i / \sigma_i$  for  $i = 1, \dots, r$ . The coordinates  $\mathbf{v}_i^\top \mathbf{b}$  for  $i > r$  are arbitrary, and the set of least squares estimates is  $\mathbf{X}^+ \mathbf{y} + \mathcal{N}(\mathbf{X})$  with  $\mathcal{N}(\mathbf{X}) = \text{span}(\mathbf{v}_{r+1}, \dots, \mathbf{v}_p)$ .
- (b)  $\mathbf{X}^+ \mathbf{y} = \sum_{i \leq r} (c_i / \sigma_i) \mathbf{v}_i$  is the unique least squares estimate of smallest Euclidean norm.
- (c)  $\hat{\mathbf{Y}} = \sum_{i \leq r} c_i \mathbf{u}_i$  and  $\text{SSE} = \sum_{i > r} c_i^2 = \|\mathbf{y}\|^2 - \sum_{i \leq r} c_i^2$ .
- (d) If  $r = p$  and  $\text{Cov}(\mathbf{Y}) = \sigma^2 \mathbf{I}$ , the estimate  $\hat{\boldsymbol{\beta}} = \mathbf{X}^+ \mathbf{Y}$  has  $\text{Cov}(\hat{\boldsymbol{\beta}}) = \sigma^2 \sum_{i \leq p} \sigma_i^{-2} \mathbf{v}_i \mathbf{v}_i^\top$ . In particular  $\text{Var}(\mathbf{v}_i^\top \hat{\boldsymbol{\beta}}) = \sigma^2 / \sigma_i^2$ , and the total variance is  $\text{tr Cov}(\hat{\boldsymbol{\beta}}) = \sigma^2 \sum_i \sigma_i^{-2}$ .

*Proof.* Let  $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_n]$ , an orthogonal matrix. For any  $\mathbf{b}$ ,  $\mathbf{X}\mathbf{b} = \sum_{i \leq r} \sigma_i (\mathbf{v}_i^\top \mathbf{b}) \mathbf{u}_i$ , so the coordinates of  $\mathbf{X}\mathbf{b}$  in the basis  $\mathbf{u}_1, \dots, \mathbf{u}_n$  are  $\sigma_i \mathbf{v}_i^\top \mathbf{b}$  for  $i \leq r$  and zero for  $i > r$ . By orthogonal invariance,

$$\|\mathbf{y} - \mathbf{X}\mathbf{b}\|^2 = \|\mathbf{U}^\top \mathbf{y} - \mathbf{U}^\top \mathbf{X}\mathbf{b}\|^2 = \sum_{i \leq r} (c_i - \sigma_i \mathbf{v}_i^\top \mathbf{b})^2 + \sum_{i > r} c_i^2. \quad (10.4.1)$$

The second sum does not involve  $\mathbf{b}$ , and the first vanishes exactly when  $\mathbf{v}_i^\top \mathbf{b} = c_i / \sigma_i$  for  $i \leq r$ . This proves (a) and (c), since  $\mathbf{b} = \sum_i (\mathbf{v}_i^\top \mathbf{b}) \mathbf{v}_i$  and  $\mathcal{N}(\mathbf{X})$  is spanned by the  $\mathbf{v}_i$  with  $i > r$  (Theorem 1.8.1). Part (b) is Proposition 6.10.1. In coordinates,  $\|\mathbf{b}\|^2 = \sum_{i \leq p} (\mathbf{v}_i^\top \mathbf{b})^2$ , so the minimum-norm estimate is the one whose free coordinates are zero,  $\sum_{i \leq r} (c_i / \sigma_i) \mathbf{v}_i$ . For (d),  $\hat{\boldsymbol{\beta}} = \sum_i \sigma_i^{-1} \mathbf{v}_i \mathbf{u}_i^\top \mathbf{Y}$ , and  $\text{Cov}(\mathbf{u}_i^\top \mathbf{Y}, \mathbf{u}_j^\top \mathbf{Y}) = \sigma^2 \mathbf{u}_i^\top \mathbf{u}_j$  is  $\sigma^2$  or 0. By Theorem 2.2.1,  $\text{Cov}(\hat{\boldsymbol{\beta}}) = \sigma^2 \sum_i \sigma_i^{-2} \mathbf{v}_i \mathbf{v}_i^\top$ . Its trace is  $\sigma^2 \sum_i \sigma_i^{-2}$  because  $\|\mathbf{v}_i\| = 1$ . ■

Part (d) is the statistical face of ill-conditioning. The combination  $\mathbf{v}_p^\top \boldsymbol{\beta}$  of the coefficients, along the direction in which the columns of  $\mathbf{X}$  are nearly dependent, is estimated with standard

deviation  $\sigma/\sigma_p$ . When  $\sigma_p$  is tiny, the data carry almost no information about that combination, and no algorithm can change this.

```
import numpy as np

def svd_ls(X, y, tol=None):
    """Minimum-norm least squares solution, treating singular values <= tol as zero.
    """
    U, s, Vt = np.linalg.svd(X, full_matrices=False)
    if tol is None:
        tol = max(X.shape) * np.finfo(float).eps * s[0]  # the usual default
    r = int(np.sum(s > tol))  # numerical rank
    c = U[:, :r].T @ y  # coordinates of the fit
    b = Vt[:r].T @ (c / s[:r])
    sse = y @ y - c @ c
    return b, r, sse, s
```

[Run this code online](#)

## 10.4.2 Computing the SVD

The SVD is not computed from the eigenvalues of  $\mathbf{X}^\top \mathbf{X}$ , which has already lost the small singular values (Section 10.1). The standard method (Golub and Kahan 1965; Golub and Reinsch 1970) first applies Householder reflections alternately from the left and the right to reduce  $\mathbf{X}$  to upper bidiagonal form, at about twice the cost of QR, and then drives the superdiagonal to zero by an implicitly shifted QR-type iteration that never forms a cross-product. When  $n \gg p$ , it pays to compute  $\mathbf{X} = \mathbf{Q}_1 \mathbf{R}$  first and then an SVD  $\mathbf{R} = \mathbf{U}_R \mathbf{\Sigma} \mathbf{V}^\top$  of the triangle, since  $\mathbf{X} = (\mathbf{Q}_1 \mathbf{U}_R) \mathbf{\Sigma} \mathbf{V}^\top$  (Exercise (B2)). Least squares needs only  $\mathbf{U}^\top \mathbf{y}$ , never  $\mathbf{U}$  itself, which gives the count  $2np^2 + 11p^3$  of Section 10.1 (Golub and Van Loan 2013).

The computed singular values are those of a nearby matrix  $\mathbf{X} + \mathbf{E}$  with  $\|\mathbf{E}\|_2 \leq cu\|\mathbf{X}\|_2$ . The next result turns this into accuracy.

### Proposition 10.4.2 — Singular values are well conditioned

For any matrices  $\mathbf{A}$  and  $\mathbf{E}$  of the same size and every  $k$ ,

$$|\sigma_k(\mathbf{A} + \mathbf{E}) - \sigma_k(\mathbf{A})| \leq \|\mathbf{E}\|_2.$$

*Proof.* For any matrix  $\mathbf{C}$  and any  $\mathbf{B}$  of rank at most  $k - 1$ ,  $\sigma_k(\mathbf{C}) \leq \|\mathbf{C} - \mathbf{B}\|_2$ . When  $k - 1 < \text{rank } \mathbf{C}$  this is the Eckart–Young inequality (Proposition 1.8.2(c)), and otherwise  $\sigma_k(\mathbf{C}) = 0$ . Let  $\mathbf{A}_{k-1}$  be the truncated SVD of  $\mathbf{A}$  with  $k - 1$  terms, or  $\mathbf{A}$  itself if  $\text{rank } \mathbf{A} < k$ , so that  $\|\mathbf{A} - \mathbf{A}_{k-1}\|_2 = \sigma_k(\mathbf{A})$ . Then

$$\sigma_k(\mathbf{A} + \mathbf{E}) \leq \|\mathbf{A} + \mathbf{E} - \mathbf{A}_{k-1}\|_2 \leq \sigma_k(\mathbf{A}) + \|\mathbf{E}\|_2.$$

Exchanging the roles of  $\mathbf{A}$  and  $\mathbf{A} + \mathbf{E}$  gives the other inequality. ■

So a backward stable SVD computes every singular value with *absolute* error of order  $u\sigma_1$ , and

a singular value of that size or below is indistinguishable from zero. An exactly rank-deficient design, such as a one-way layout with an intercept and all its indicators, therefore shows a smallest singular value near  $10^{-16}\sigma_1$  rather than zero:  $5 \times 10^{-17}$  times the largest in the script's example.

### 10.4.3 Numerical rank

In floating point, and with data measured to finite precision, the meaningful question is how many singular values are *clearly* nonzero.

#### Definition 10.4.1 — Numerical rank

For a tolerance  $\tau > 0$ , the **numerical rank** of  $\mathbf{X}$  is  $r_\tau = \#\{i : \sigma_i > \tau\}$ . The **truncated SVD** with  $k$  terms is  $\mathbf{X}_k = \sum_{i \leq k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ .

By the Eckart–Young inequality,  $r_\tau$  is the least rank of any matrix within spectral distance  $\tau$  of  $\mathbf{X}$  (Exercise (B1)), so  $\tau$  should reflect how well  $\mathbf{X}$  is known. The default in numpy and LAPACK-based software,  $\tau = \max(n, p) \epsilon_M \sigma_1$ , accounts for rounding only; data recorded to six significant figures cannot resolve singular values below about  $10^{-6}\sigma_1$ . The choice matters, because the minimum-norm solution is not a continuous function of  $\mathbf{X}$ .

#### Example. (The pseudoinverse jumps)

Let  $\mathbf{x}_1 = (1, 2, 3, 4)^\top$ , let  $\mathbf{d} = (1, -1, -1, 1)^\top$  be orthogonal to it, and let  $\mathbf{X}_t = [\mathbf{x}_1, \mathbf{x}_1 + t\mathbf{d}]$ , with  $\mathbf{y} = (1, 3, 2, 5)^\top$ . For every  $t \neq 0$  the column space is the same plane  $\text{span}(\mathbf{x}_1, \mathbf{d})$ , so the fitted values do not depend on  $t$ . The coefficients do. At  $t = 10^{-6}$  they are  $-249999$  and  $250000$ , because the coefficients must reproduce a fixed multiple of  $\mathbf{d}$  using the small difference  $t\mathbf{d}$  of the two columns. At  $t = 0$  the rank drops to one. The minimum-norm solution jumps to  $0.550$  for both columns, and the fitted values jump too, because the  $\mathbf{d}$ -direction has disappeared from the model. Treating singular values below a tolerance as zero makes the two answers agree. Whether it should is a statistical question: is the difference between the columns signal, or recording error?

```
x1 = np.array([1.0, 2.0, 3.0, 4.0])
d = np.array([1.0, -1.0, -1.0, 1.0])          # orthogonal to x1
yd = np.array([1.0, 3.0, 2.0, 5.0])
for t in [1e-2, 1e-6, 1e-10, 0.0]:
    Xt = np.column_stack([x1, x1 + t * d])    # second column tends to the first
    b = np.linalg.pinv(Xt, rcond=1e-15) @ yd
    print(f"t = {t:7.0e}    b = {np.round(b, 3)}    fit = {np.round(Xt @ b, 3)}")
```

[Run this code online](#)

### 10.4.4 Truncated SVD as an estimator

The truncated solution  $\mathbf{b}_k = \sum_{i \leq k} (c_i / \sigma_i) \mathbf{v}_i$  is more than a numerical safeguard. It is a statistical estimator in its own right, and it trades bias for variance.

**Proposition 10.4.3 — Bias and variance of truncated SVD**

Suppose  $\text{rank } \mathbf{X} = p$ ,  $E(\mathbf{Y}) = \mathbf{X}\boldsymbol{\beta}$  and  $\text{Cov}(\mathbf{Y}) = \sigma^2 \mathbf{I}$ , and let  $\mathbf{b}_k = \sum_{i \leq k} (\mathbf{u}_i^\top \mathbf{Y} / \sigma_i) \mathbf{v}_i$  for  $k \leq p$ . Then

$$E(\mathbf{b}_k) = \sum_{i \leq k} (\mathbf{v}_i^\top \boldsymbol{\beta}) \mathbf{v}_i, \quad \text{Cov}(\mathbf{b}_k) = \sigma^2 \sum_{i \leq k} \frac{\mathbf{v}_i \mathbf{v}_i^\top}{\sigma_i^2},$$

$$E\|\mathbf{b}_k - \boldsymbol{\beta}\|^2 = \sum_{i > k} (\mathbf{v}_i^\top \boldsymbol{\beta})^2 + \sigma^2 \sum_{i \leq k} \frac{1}{\sigma_i^2}. \quad (10.4.2)$$

Hence  $\mathbf{b}_k$  has smaller mean squared error than the least squares estimate  $\mathbf{b}_p$  iff  $\sum_{i > k} (\mathbf{v}_i^\top \boldsymbol{\beta})^2 < \sigma^2 \sum_{i > k} \sigma_i^{-2}$ .

*Proof.*  $E(\mathbf{u}_i^\top \mathbf{Y}) = \mathbf{u}_i^\top \mathbf{X}\boldsymbol{\beta} = \sigma_i \mathbf{v}_i^\top \boldsymbol{\beta}$ , which gives the mean. The covariance is part (d) of [Theorem 10.4.1](#) with the sum stopped at  $k$ . For the mean squared error, write  $E\|\mathbf{b}_k - \boldsymbol{\beta}\|^2 = \|E\mathbf{b}_k - \boldsymbol{\beta}\|^2 + \text{tr Cov}(\mathbf{b}_k)$ . The bias  $E\mathbf{b}_k - \boldsymbol{\beta}$  is  $-\sum_{i > k} (\mathbf{v}_i^\top \boldsymbol{\beta}) \mathbf{v}_i$ , with squared length  $\sum_{i > k} (\mathbf{v}_i^\top \boldsymbol{\beta})^2$ . Comparing [Eq. 10.4.2](#) for  $k$  and  $p$  gives the last claim. ■

Each dropped component costs its squared coefficient  $(\mathbf{v}_i^\top \boldsymbol{\beta})^2$  in bias and saves  $\sigma^2/\sigma_i^2$  in variance. It is worth dropping exactly when the signal along  $\mathbf{v}_i$  is weaker than the noise in its estimate, that is, when  $\sigma_i |\mathbf{v}_i^\top \boldsymbol{\beta}| < \sigma$ . The estimator  $\mathbf{b}_k$  is principal components regression in its uncentred form. Ridge regression replaces the hard cut-off by weights  $\sigma_i^2/(\sigma_i^2 + \lambda)$ . Both are developed in [Chapter 27](#) (Theorem 27.2.2 and Definition 27.3.1), and choosing  $k$  from data is a model-selection problem ([Chapter 29](#)).

**Example. (Truncating a polynomial design)**

The powers  $1, t, \dots, t^7$  at 60 equally spaced points in  $[0, 1]$  form a design with  $\kappa(\mathbf{X}) = 1.1 \times 10^5$  and smallest singular value  $9.0 \times 10^{-5}$ . Take  $\sigma = 0.1$  and a coefficient vector whose components along  $\mathbf{v}_1, \dots, \mathbf{v}_8$  decay  $(2, -1.5, 1, -0.6, 0.3, -0.1, 0.05, -0.02)$ , as they do for smooth regression functions. By [Eq. 10.4.2](#), least squares has mean squared error 1231055. The best truncation keeps 4 components and has mean squared error 0.21, made up of squared bias 0.10 and variance 0.11. A simulation with 4000 replicates agrees with the formula at every  $k$  ([Figure 10.4.1](#)). The default numerical tolerance treats the design as having full rank, which it does to working precision. The statistical “effective rank” is set by the noise level instead.

```
n, p, sigma = 60, 8, 0.1
t = np.linspace(0, 1, n)
X = np.vander(t, p, increasing=True)          # 1, t, ..., t^7: badly conditioned
U, s, Vt = np.linalg.svd(X, full_matrices=False)
a = np.array([2.0, -1.5, 1.0, -0.6, 0.3, -0.1, 0.05, -0.02]) # decaying components
      v_i' beta
beta = Vt.T @ a
print("condition number:", s[0] / s[-1])
print("singular values:", np.array2string(s, precision=2))
```

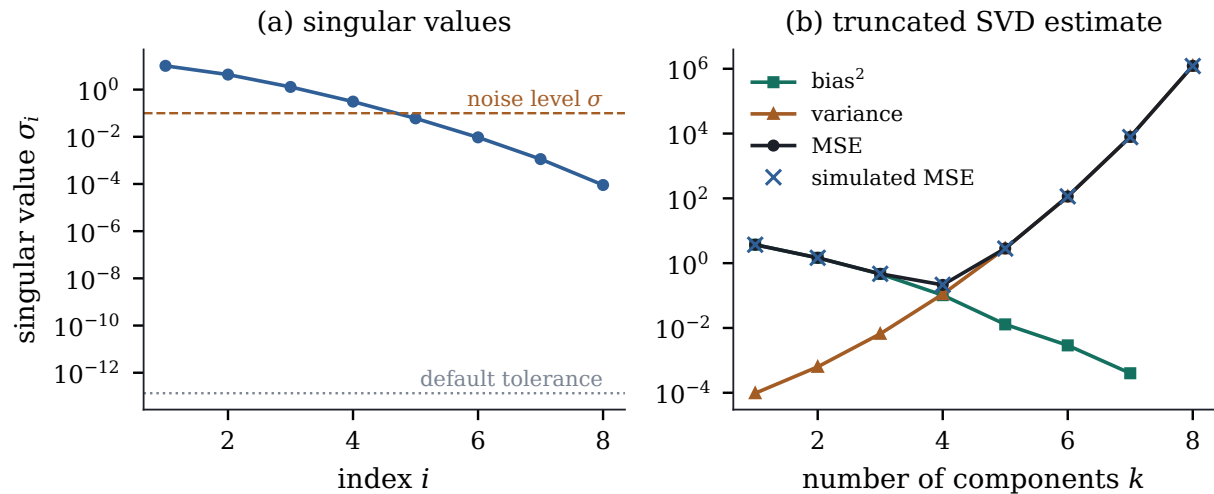


Figure 10.4.1: Truncated SVD for a degree-7 polynomial design. (a) Singular values, with the default numerical-rank tolerance and the noise level  $\sigma$ . (b) Squared bias, variance and mean squared error of the truncated estimate  $\mathbf{b}_k$  (Eq. 10.4.2), with simulated values. Least squares is  $k = 8$ .

[Run this code online](#)

```
bias2 = np.array([np.sum((Vt[k:] @ beta) ** 2) for k in range(1, p + 1)]) # //(I -
                    V_k V_k') beta//^2
var = np.array([sigma ** 2 * np.sum(1 / s[:k] ** 2) for k in range(1, p + 1)])
mse = bias2 + var
print("k    bias^2    variance    MSE")
for k in range(p):
    print(f"{k + 1}  {bias2[k]:10.3e}  {var[k]:10.3e}  {mse[k]:10.3e}")
```

[Run this code online](#)

*Idea.* The SVD gives every least squares solution, the precision  $\sigma/\sigma_i$  of each combination  $\mathbf{v}_i^\top \boldsymbol{\beta}$ , and, relative to a tolerance, how many combinations are determined at all.

## 10.4.5 Exercises

### 10.4.5.1 A. Check your understanding

#### Exercise A1.

Let  $\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}$  and  $\mathbf{y} = (1, 3, 5)^\top$ . Find the SVD of  $\mathbf{X}$ , the set of least squares estimates, the minimum-norm estimate and the SSE.

*Solution.*  $\mathbf{X} = 2\mathbf{u}_1\mathbf{v}_1^\top$  with  $\mathbf{u}_1 = (1, 1, 0)^\top/\sqrt{2}$ ,  $\mathbf{v}_1 = (1, 1)^\top/\sqrt{2}$ ,  $\sigma_1 = 2$  and  $r = 1$ . Here  $c_1 = \mathbf{u}_1^\top \mathbf{y} = 4/\sqrt{2}$ , so the least squares estimates are the  $\mathbf{b}$  with  $\mathbf{v}_1^\top \mathbf{b} = c_1/\sigma_1 = \sqrt{2}$ , that is,  $b_1 + b_2 = 2$ . The minimum-norm estimate is  $\sqrt{2}\mathbf{v}_1 = (1, 1)^\top$ . The fitted vector is  $c_1\mathbf{u}_1 = (2, 2, 0)^\top$ , and  $\text{SSE} = \|\mathbf{y}\|^2 - c_1^2 = 35 - 8 = 27$ , which is  $(1-2)^2 + (3-2)^2 + 5^2$ .

### 10.4.5.2 B. Practice

**Exercise B1.**

Show that if  $\tau$  is not a singular value of  $\mathbf{X}$ , then  $r_\tau = \min\{\text{rank } \mathbf{B} : \|\mathbf{X} - \mathbf{B}\|_2 \leq \tau\}$ , and that the minimum is attained by the truncated SVD  $\mathbf{X}_{r_\tau}$ .

*Solution.*  $\|\mathbf{X} - \mathbf{X}_k\|_2 = \sigma_{k+1}$ . For  $k = r_\tau$ ,  $\sigma_{k+1} \leq \tau$  (with  $\sigma_{k+1} = 0$  if  $k = \text{rank } \mathbf{X}$ ), so  $\mathbf{X}_{r_\tau}$  is within  $\tau$  and has rank  $r_\tau$ . If  $\text{rank } \mathbf{B} = k < r_\tau$ , then by Proposition 1.8.2(c)  $\|\mathbf{X} - \mathbf{B}\|_2 \geq \sigma_{k+1} \geq \sigma_{r_\tau} > \tau$ . So no matrix of smaller rank is within  $\tau$ .

**Exercise B2.**

Let  $\mathbf{X} = \mathbf{Q}_1 \mathbf{R}$  be a thin QR and  $\mathbf{R} = \mathbf{U}_R \mathbf{\Sigma} \mathbf{V}^\top$  an SVD of the triangle. Show that  $\mathbf{X} = (\mathbf{Q}_1 \mathbf{U}_R) \mathbf{\Sigma} \mathbf{V}^\top$  is an SVD of  $\mathbf{X}$ , and that the minimum-norm least squares solution can be computed from  $\mathbf{R}$  and  $\mathbf{Q}_1^\top \mathbf{y}$  alone. Why is this attractive when  $n$  is much larger than  $p$ ?

**Exercise B3.**

Show that  $\mathbf{b}_k$  minimizes  $\|\mathbf{y} - \mathbf{X}\mathbf{b}\|$  over  $\mathbf{b} \in \text{span}(\mathbf{v}_1, \dots, \mathbf{v}_k)$ , and that it is the minimum-norm least squares estimate for the matrix  $\mathbf{X}_k$ . Interpret  $\mathbf{b}_k$  as least squares after deleting the directions in which  $\mathbf{X}$  is nearly degenerate.

**10.4.5.3 C. Going deeper****Exercise C1.**

Show that the ridge estimate  $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$  equals  $\sum_i \frac{\sigma_i}{\sigma_i^2 + \lambda} c_i \mathbf{v}_i$ . Derive its bias and variance in the form of Proposition 10.4.3, and show that for every  $\boldsymbol{\beta}$  there is a  $\lambda > 0$  (depending on  $\boldsymbol{\beta}$  and  $\sigma^2$ ) for which its mean squared error is below that of least squares.